MALLA REDDY UNIVERSITY

A Major Project Phase-1 Project Report

Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence

Design & Developed by:

A.Nikhitha	2311CS040013
K.Sanjana	2311CS040083
K.Hemanth	2311CS040086

GUIDED BY

Dr. M. V. N. Srujan Manohar

Assistant Professor

Department of Computer Science & Engineering (Cyber Security)

B.Tech IV Year - I Semester

Malla Reddy University, Hyderabad

October -2026

MALLA REDDY UNIVERSITY

(Telangana State Private Universities Act No.13 of 2020 and G.O.Ms.No.14, Higher Education (UE) Department)

Department of Computer Science and Engineering (Cyber Security)

CERTIFICATE

This is to certify that this is the Bonafide record of the Major Project Phase-1 project entitled **"Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence"** submitted by **A.Nikhitha(2311CS04013), K.Sanjana(2311CS040083), K.Hemanth (2311CS040086),** of **B.Tech IV Year I semester**, Department of CSE(Cyber Security) during the academic year 2026-2027.The results embodied in this report have not been submitted to any other university or institute for award of any degree or diploma.

Internal Guide

Dr. M. V. N. Srujan Manohar

Assistant Professor

Dean&HOD (Cyber security)

Dr. G. Anand Kumar

Professor & Dean

External Examiner

DECLARATION

We hereby declare that the project report entitled **"Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence"** has been carried out by us and submitted to the Department of Computer Science and Engineering (Cyber Security), Malla Reddy University, Hyderabad. This project work has been designed, developed & deployed by us, as part of the Application Development Project Work of this semester. We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

Place:

Date:

A.Nikhitha	2311CS040013
K.Sanjana	2311CS040083
K.Hemanth	2311CS040086

ACKNOWLEDGEMENT

We extend our sincere gratitude to all those who have contributed to the completion of this project report. Firstly, we would like to extend our gratitude **to Dr. V. S. K Reddy,** Vice Chancellor**,** for his visionary leadership and unwavering commitment to academic excellence.

We are also grateful **to Dr. G. Anand Kumar,** Dean (Cyber Security), for providing us with the necessary resources and facilities to carry out this project.

We would also like to express my deepest appreciation to our project guide **Dr. M. V. N. Srujan Manohar,** Assistant Professor**,** whose invaluable guidance, insightful feedback, and unwavering support have been instrumental throughout the course of this project for successful outcomes.

We extend our gratitude to our **PRC – convenor, Dr. G. Latha,** for giving valuable inputs and timely guidelines to improve the quality of our project through a critical review process. We extend our gratitude to our **Project Coordinator**, **Mr. A. Ramesh Khanna,** for his valuable support in review process.

We are deeply indebted to all of them for their support, encouragement, and guidance, without which this project would not have been possible.

A.Nikhitha 2311CS040013

K.Sanjana 2311CS040083

K.Hemanth 2311CS040086

ABSTRACT

The increasing use of the Internet has led to a rapid rise in cyber threats such as phishing, malware and malicious websites. These attacks are designed to steal personal information, distribute harmful software and compromise user security. Traditional URL detection techniques mainly depend on blacklist databases and predefined rules, which are often unable to identify newly created malicious websites. Therefore, there is a need for an intelligent and adaptive system that can accurately detect malicious links in real time.

This project presents a Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence. The framework combines Machine Learning, Deep Learning and Agentic AI to improve the accuracy and efficiency of malicious URL detection. Important features such as URL length, domain characteristics, HTTPS usage, suspicious keywords and subdomains are extracted from the input URL. These features are processed by a Random Forest model and a Deep Learning model to classify URLs into four categories: Benign, Phishing, Malware and Defacement.

An Agentic AI architecture is incorporated in which multiple intelligent agents perform specific tasks such as URL validation, feature extraction, threat detection, threat intelligence analysis and report generation, coordinated by an Orchestrator Agent. After classification, the framework provides the prediction result, confidence score, risk level and security recommendations to help users make informed decisions before accessing a website.

Keywords: *Hybrid Deep Learning, Agentic AI, Malicious Link Detection, Web Threat Intelligence, Phishing Website Detection, Machine Learning, Cybersecurity, URL Classification.*

INDEX

	Contents	Page No.
	Chapter 1 Introduction	**1 - 3**
1.1	*Problem Definition & Description*	1
1.2	*Aim and Objectives of the Project*	2
1.3	*Scope of the Project*	3
	Chapter 2 System Analysis	**4 - 10**
2.1	*Existing System Analysis*	4
2.1.1	*Background & Literature Survey*	5
2.1.2	*Limitations of Existing System*	6
2.2	*Proposed System Analysis*	6
2.2.1	*Proposed System Description*	8
2.2.2	*Advantages of Proposed System*	8
2.3	*Software & Hardware Requirements*	9
2.3.1	*Software Requirements*	9
2.3.2	*Hardware Requirements*	9
2.4	*Feasibility Study*	10
	Chapter 3 Architectural Design	**11-20**
3.1	*Modules Design*	11
3.1.1	*Module 1: Orchestrator Agent*	11
3.1.2	*Module 2: URL Validation Agent*	12
3.1.3	*Module 3: Feature Extraction Agent*	12
3.1.4	*Module 4: Machine and deep learning Agents*	12
3.1.5	*Module 5: Report Generation Agents*	12
3.1.6	*Module 6: Web Application Modules*	12
3.2	*Methodology & Algorithm Design*	13
3.2.1	*Method 1: URL Validation & Feature Extraction*	14
3.2.2	*Method 2: Hybrid Detection & Threat Analysis*	14
3.2.3	*Algorithm 1: Random forest classifier*	14
3.2.4	*Algorithm 2: 1D-CNN + BiLSTM Deep learning Algorithm*	14

3.3 *System Architecture* 15

3.3.1 *Architectural Diagram* 15

3.3.2 *Class Diagram* 16

3.3.3 *Use Case Diagram* 18

3.3.4 *Sequence Diagram* 19

3.3.5 *Deployment Diagram* 20

Chapter 4 Implementation & Testing **21 - 27**

4.1 *Coding Blocks* 21

4.1.1 *Main Code Block – Flask Application and Agent Pipeline* 21

4.1.2 *Functional Code Block 1 – URL Validation and Feature Extraction* 22

4.1.3 *Functional Code Block 2 – Model Training and Hybrid Decision* 23

4.1.4 *Code Execution Flow Chart* 24

4.2 *Test Case Study* 27

4.2.1 *Test Case 1: Valid Benign URL* 27

4.2.2 *Test Case 2: Malicious URL (Phishing / Malware)* 27

Chapter 5 Results **28- 32**

5.1 *Resulting Images* 28 - 29

5.2 *Resulting Tables* 30

5.2.1 *Table 1: Performance Metrics of Proposed Framework* 30

5.2.2 *Table 2: Comparison of Detection Approaches* 30

5.3 *Resulting Graphs* 31

5.3.1 *Graph 1: Performance Metrics* 31

5.3.2 *Graph 2: Threat Severity Distribution* 31

5.4 Results Analysis 32

Chapter 6 Conclusions & Future Scope **33**

6.1 *Conclusions* 33

6.2 *Future Scope* 33

References 34

Paper Draft 35 - 40

Poster 41

List of Diagrams

Contents		***Page No.***
Figure 3.2.1	*Methodology*	15
Figure 3.3.1	*Architectural Diagram*	15
Figure 3.3.3	*Class Diagram*	16
Figure 3.3.4	*Use Case Diagram*	18
Figure 3.3.5	*Sequence Diagram*	19
Figure 3.3.6	*Deployment Diagram*	20

List of Tables

Contents		***Page No.***
Table 5.2.1	*Performance Metrics of proposed Framework*	30
Table 5.2.2	*Comparison of Detection Approaches*	30

List of Graphs

Contents	***Page No.***
Graph 5.3.1 Performance Metrics	31
Graph 5.3.2 Threat Severity Distribution	31

List of Screenshots

Contents	***Page No.***
Screenshot 5.1.1 Login Page	28
Screenshot 5.1.2 Prediction /Threat Scanner Page	28
Screenshot 5.1.3 Dashboard	29

CHAPTER - 1

INTRODUCTION

1.1 PROBLEM DEFINITION & DESCRIPTION

The rapid growth of the Internet has increased the use of websites and online services, but it has also led to a rise in cyber threats such as phishing, malware, and website defacement. Malicious links are often designed to deceive users into visiting harmful websites, downloading malicious content, or revealing sensitive information. Traditional security methods, such as blacklist-based and rule-based systems, are useful for detecting known threats but may fail to identify newly created or previously unseen malicious URLs.

The proposed project, **"Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence,"** aims to develop an intelligent framework for detecting malicious URLs using a combination of Machine Learning, Deep Learning, and Agentic AI. The system analyzes different URL characteristics such as URL length, domain properties, HTTPS usage, suspicious keywords, subdomains, special characters, and other structural features.

The framework uses Random Forest and a Deep Learning model to classify URLs into four categories: Benign, Phishing, Malware, and Defacement. Agentic AI components automate important tasks such as URL validation, feature extraction, prediction, threat analysis, and report generation. The system provides the user with the predicted category, confidence score, risk level, and recommended security actions.

The system classifies URLs into four categories:

- **Benign** – Safe or legitimate URLs.
- **Phishing** – URLs designed to deceive users and obtain sensitive information.
- **Malware** – URLs associated with malicious software or harmful content.
- **Defacement** – URLs related to compromised or altered websites.

The framework is designed to provide not only a prediction but also confidence scores, risk levels, threat analysis, and security recommendation.

1.2 OBJECTIVES AND SCOPE

The main objective of the proposed project is to develop an intelligent and adaptive framework for detecting and analysing malicious URLs using Machine Learning, Deep Learning, and Agentic AI techniques. The system is designed to identify potentially harmful links and provide users with meaningful security information before they access them. The major objectives of the project are as follows:

1. **To detect malicious URLs:** Develop an automated system that can analyze a given URL and determine whether it is safe or potentially harmful.
2. **To classify different types of threats:** Classify URLs into four categories: Benign, Phishing, Malware, and Defacement, allowing different types of web threats to be identified.
3. **To extract URL features:** Analyze important lexical, structural, domain-based, and security-related characteristics of URLs, such as URL length, number of subdomains, special characters, HTTPS usage, suspicious keywords, and URL entropy.
4. **To improve detection accuracy:** Combine a Random Forest model and a Deep Learning model to improve classification performance and reduce incorrect classifications.
5. **To automate threat detection:** Use Agentic AI components to automate important tasks such as URL validation, feature extraction, prediction, threat analysis, and report generation.
6. **To provide risk assessment:** Generate confidence scores and risk levels for detected URLs so that users can better understand the severity of a potential threat.
7. **To provide security recommendations:** Generate appropriate recommendations and suggested actions based on the detected URL category and associated risk.
8. **To support previously unseen threats:** Develop an adaptive approach that can identify malicious URL patterns without depending completely on traditional blacklist databases.
9. **To develop a user-friendly web application:** Provide a Flask-based interface where users can submit URLs and receive real-time prediction and threat-analysis results.
10. **To generate automated security reports:** Present the URL, predicted category, confidence score, risk level, threat information, and recommended actions in a structured report.

Overall, the project aims to provide a reliable, automated, and scalable malicious-link detection framework that combines hybrid machine learning with Agentic AI to improve web security and help users make safer decisions when interacting with online links.

1.3 SCOPE OF THE PROJECT

The scope of the project includes the following major areas:

- **URL Classification:** Detection of URLs belonging to Benign, Phishing, Malware, and Defacement categories.
- **Feature Extraction:** Analysis of URL length, domain length, dots, hyphens, digits, subdomains, HTTPS usage, IP addresses, special characters, suspicious keywords, query parameters, URL entropy, and domain age when available.
- **Machine Learning:** Use of a Random Forest classifier for URL classification.
- **Deep Learning:** Use of a fully connected neural network for learning complex URL patterns.
- **Hybrid Decision Making:** Combining the outputs of the Random Forest and Deep Learning models to produce the final prediction.
- **Agentic AI:** Use of specialized agents for validation, feature extracti on, prediction, threat intelligence, and report generation.
- **Threat Intelligence:** Generation of confidence scores, risk levels, threat analysis, and recommended actions.
- **Web Application:** Development of a Flask-based web application with a user-friendly interface.
- **Dashboard and Analytics:** Providing users with prediction results and analysis through the web application.
- **Real-Time Scanning:** Allowing users to submit and analyze individual URLs.

CHAPTER - 2

SYSTEM ANALYSIS

System analysis studies the existing methods used for detecting malicious URLs. Traditional systems mainly depend on blacklist databases and rule-based techniques. Machine Learning methods analyze URL features to improve detection accuracy. However, these systems may fail to detect newly created or unknown malicious URLs. They also provide limited threat analysis and often require frequent database updates. Therefore, an intelligent and adaptive system is required for better real-time protection.

2.1 EXISTING SYSTEM ANALYSIS

Existing malicious URL detection systems primarily rely on blacklist databases maintained by web browsers, antivirus solutions, and cybersecurity organizations. In these systems, the URL provided by a user is compared against a database of previously identified malicious or suspicious websites. Machine Learning techniques such as Decision Trees, Naive Bayes, Logistic Regression, Support Vector Machines, and Random Forest are also used to analyze URL-based features and classify websites. These approaches have improved the accuracy of malicious URL detection and provide an effective method for identifying known threats.

However, existing systems have several limitations when dealing with newly created, modified, or previously unseen malicious URLs. Their dependence on regularly updated blacklist databases makes them less effective against emerging threats and unknown attack patterns. Many systems also rely on manually engineered features and mainly focus on URL classification without providing detailed threat analysis or intelligent security recommendations. As a result, they may produce false positives or fail to identify sophisticated and evolving web-based attacks, creating a need for a more adaptive and intelligent malicious URL detection framework.

2.1.1 BACKGROUND AND LITERATURE SURVEY

1. Machine Learning-Based Malicious URL Detection

Machine Learning has been widely applied to malicious URL detection by extracting lexical and host-based features from URLs. Different algorithms such as Random Forest, Decision Tree, Support Vector Machine (SVM), and Logistic Regression have been used to classify URLs. Random Forest provides robust performance because of its ensemble learning capability and reduced overfitting. However, many existing approaches depend on manually engineered features and may have difficulty detecting newly emerging attack patterns.

Research Reference:

[2] A. Nandu, J. Sosa, Y. Pant, Y. Panchal, and S. Sayyad, "Malicious URL Detection Using Machine Learning," in Proc. 4th IEEE Asian Conf. Innovation in Technology (ASIANCON), 2024.

2. Deep Learning-Based Malicious URL Detection

Deep Learning techniques provide an advanced method for identifying malicious URLs by learning complex patterns from URL data. These approaches can improve the detection of phishing and other malicious websites by learning representations that may not be easily captured through manually designed features. Deep Learning models are particularly useful for detecting complex and evolving URL patterns.

Research Reference:

[11] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, "Lightweight Malicious URL Detection Using Deep Learning and Large Language Models," Scientific Reports, vol. 15, Art. no. 43044, 2025.

3. Hybrid and Sequential Deep Learning Approaches

Hybrid approaches combine different machine learning and deep learning techniques to improve malicious URL detection. Such approaches can benefit from both feature-based learning and automatic representation learning, potentially improving accuracy and reducing false positives. Sequential deep learning models can also use contextual information from URLs to improve phishing website detection.

Research Reference:

[10] J. Zhou, K. Zhang, A. Bilal, Y. Zhou, Y. Fan, W. Pan, X. Xie, and Q. Peng, "An Integrated CSPPC and BiLSTM Framework for Malicious URL Detection," Scientific Reports, vol. 15, Art. no. 6659, 2025.

4. Web Threat Intelligence for Malicious URL Detection

Web Threat Intelligence techniques analyze different characteristics of URLs and domains, including domain information, HTTPS usage, suspicious keywords, IP addresses, and URL structures. These techniques help identify potential threats and provide additional information for security analysis. However, systems that depend heavily on continuously updated external databases may have limitations when dealing with newly created phishing and malware websites.

Research Reference:

[17] L. Chen and L. Meng, "Metadata Driven Malicious URL Detection Using RoBERTa-Large and Multi-Source Network Threat Intelligence," Scientific Reports, vol. 16, Art. no. 5449, 2026

2.1.2 LIMITATIONS OF EXISTING SYSTEM

1. **Unable to Detect Newly Generated URLs:** Traditional blacklist and signature-based systems mainly identify previously reported malicious URLs and may fail to detect newly created or zero-day malicious websites.
2. **Dependence on Blacklist Databases:** Existing systems depend heavily on continuously updated blacklist databases. If a malicious URL is not present in the database, it may not be identified correctly.
3. **Limited Feature Representation:** Systems that depend on manually engineered URL features may not capture complex patterns associated with sophisticated and evolving cyber attacks.
4. **High False Positive and False Negative Rates:** Single-model detection approaches may incorrectly classify legitimate URLs as malicious or fail to identify some malicious URLs, affecting the reliability of the system.

5. **Lack of Intelligent Threat Analysis:** Many existing systems focus mainly on URL classification and do not provide automated threat analysis, risk scoring, security recommendations.
6. **Frequent Updates and Retraining:** Changing attack patterns require frequent model retraining and manual database updates, which reduces the adaptability of traditional detection systems.
7. **Limited Real-Time Efficiency:** Searching large and continuously updated databases can increase the detection time and reduce the efficiency of real-time malicious URL detection.

2.2 PROPOSED SYSTEM ANALYSIS

2.2.1 PROPOSED SYSTEM DESCRIPTION

The proposed system is a Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence. It combines Machine Learning, Deep Learning, and Agentic AI to intelligently detect malicious URLs. The system accepts a URL through a Flask-based web application and first validates the input. It then extracts important lexical, domain-based, and security-related features from the URL. These features are processed by a Random Forest model and a Deep Learning neural network. The predictions from both models are combined using a Hybrid Decision Agent to improve classification reliability. The system classifies URLs into four categories: Benign, Phishing, Malware, and Defacement. The Threat Intelligence Agent analyzes the prediction and generates a confidence score, risk score, threat level, and security recommendations. Finally, the Report Generation Agent prepares and displays the complete security analysis to the user.

2.2.2 ADVANTAGES OF PROPOSED SYSTEM

The proposed system provides several advantages over conventional malicious URL detection approaches:

1. High Detection Accuracy

The combination of Random Forest and Deep Learning improves the reliability of URL

classification. The experimental results reported in the project show an accuracy of 98.7%, with 98.0% precision, 97.0% recall, and 98.0% F1-scor

2. Detection of Previously Unseen URLs

Unlike systems that depend entirely on previously recorded blacklist entries, the proposed framework learns patterns from URL features. This allows it to identify potentially malicious URLs based on their characteristics, including URLs that may not already exist in a blacklist.

3. Hybrid Prediction

The system combines the predictions of Random Forest and Deep Learning rather than depending on a single model. The Hybrid Decision Agent evaluates the outputs and combines their probability values to determine the final class and confidence score. This helps reduce the effect of errors from an individual model.

4. Automated Threat Analysis

The Agentic AI architecture automates multiple stages of the security process, including URL validation, feature extraction, prediction, threat analysis, and report generation. This reduces the amount of manual intervention required from the user or security analyst.

5. Risk Assessment and Security Recommendations

The system does more than simply label a URL as malicious or benign. It generates a confidence score, risk score, threat level, and security recommendations, giving users additional information about the detected threat and recommended action.

6. Four-Class Threat Classification

The framework provides more detailed classification by categorizing URLs into Benign, Phishing, Malware, and Defacement. This provides more useful information than a simple safe/unsafe classification.

7. Modular and Scalable Architecture

The system is divided into independent agents and modules. Each component performs a specific

task and can be improved or modified independently. This modular design makes the framework easier to maintain and provides scope for future enhancements.

2.3 SOFTWARE & HARDWARE REQUIREMENTS

2.3.1 SOFTWARE REQUIREMENTS

- Operating System: Windows 10/11
- Programming Language: Python 3.x
- Frontend: HTML, CSS, JavaScript
- Backend Framework: Flask
- Machine Learning Library: Scikit-learn (Random Forest)
- Deep Learning Library: TensorFlow (Neural Network)
- Dataset: Malicious URLs Dataset (Kaggle)
- IDE: Visual Studio Code
- Browser: Any modern web browser

2.3.2 HARDWARE REQUIREMENTS

- Processor: Intel Core i5 or higher
- RAM: 8 GB or higher
- Storage: Sufficient disk space for the dataset and trained models
- Network: Internet connection for package installation and web access

2.4 FEASIBILITY STUDY

The proposed **Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence** is feasible because it uses mature and freely available technologies such as Python, Flask, Scikit-learn, and TensorFlow. The project can run on a standard laptop with an Intel Core i5 processor and 8 GB RAM, and the required malicious URL dataset is publicly available. The extracted URL features are lightweight, making the system technically feasible to develop, train, and deploy. From an economic point of view, the software tools and dataset are free and open source, so no special hardware is required and the overall development cost is minimal.

The feasibility study is categorized into three key aspects:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

2.4.1 Technical Feasibility

The project is technically feasible because it can run on a standard laptop with an Intel Core i5 processor and 8 GB RAM. It uses established technologies such as Python, Flask, Scikit-learn, and TensorFlow. The extracted URL features are lightweight, making the system suitable for development, training, and deployment.

2.4.2 Robustness & Reliability Feasibility

The system combines Random Forest and Deep Learning models to detect malicious URLs. Its reliability depends on data quality, model performance, and testing with unseen URLs. It can classify URLs into Benign, Phishing, Malware, and Defacement categories. Regular dataset updates and proper error handling help improve detection performance and ensure consistent results.

2.4.3 Economic Feasibility

The project is economically feasible because it uses free and open-source technologies such as Python, Flask, Scikit-learn, and TensorFlow. The malicious URL dataset is publicly available, and no special or expensive hardware is required. Therefore, the development and implementation cost of the project is minimal.

CHAPTER - 3
ARCHITECTURAL DESIGN

3.1 MODULES DESIGN

The framework consists of an Orchestrator Agent and seven functional modules/agents. Each module is described below.

3.1.1 Orchestrator Agent

The central controller of the framework. It manages communication between all agents, coordinates execution of each task in the correct sequence, forwards the user's URL to the URL Validation Agent, monitors execution and ensures the output of each agent is passed to the next stage.

3.1.2 URL Validation Agent

Verifies the correctness and validity of the input URL: whether it follows the standard format, contains a valid domain and uses an appropriate protocol. Invalid or malformed URLs are rejected; valid URLs are forwarded to the Feature Extraction Agent. This reduces unnecessary computation and improves reliability.

S.No	Feature	Description
1	URL Length	Total number of characters in the URL
2	Domain Length	Length of the domain name
3	Number of Dots	Count of '.' characters
4	Number of Hyphens	Count of '-' characters
5	Number of Digits	Count of numeric characters
6	Number of Subdomains	Subdomain levels in the host
7	HTTPS Usage	Whether the URL uses HTTPS
8	Presence of IP Address	IP address used in place of domain
9	Special Characters	Count of special characters
10	Suspicious Keywords	Words such as login, verify, secure, update, bank
11	Slash Count	Number of '/' characters
12	Query Parameters	Number of query parameters
13	URL Entropy	Randomness (Shannon entropy) of the URL
14	Domain Age	Age of the domain, if available

Table 3.1 Extracted URL features

3.1.3 Feature Extraction Agent

Extracts lexical, domain-based and security-related features from the validated URL (Table 3.1) and converts them into a numerical feature vector that serves as input to the Machine Learning and Deep Learning models.

3.1.4 Machine Learning and Deep Learning Agents

The Machine Learning Agent classifies the feature vector using the Random Forest algorithm, which builds multiple decision trees and combines their predictions to assign one of the four classes. The Deep Learning Agent processes the same vector with a fully connected neural network having multiple hidden layers to learn hidden feature representations and complex attack patterns that traditional algorithms may miss. Both agents forward their predictions to the Hybrid Decision Agent.

3.1.5 Hybrid Decision, Threat Intelligence and Report Generation Agents

The Hybrid Decision Agent compares the outputs of the two models, evaluates their confidence scores and determines the final classification, improving accuracy and reducing false positives. The Threat Intelligence Agent analyzes the final prediction, calculates the confidence score and risk score, assigns the risk/threat level and generates security recommendations. The Report Generation Agent prepares a security report containing the URL, prediction category, confidence score, risk score, threat level and recommended actions, and displays it through the web application.

3.1.6 Web Application Module

Developed using HTML, CSS and JavaScript for the front end and Flask for the back end. It provides Home, Login, Scan Console (threat scanner), Dashboard, Analytics and About/Contact pages, and displays the prediction result, confidence score, risk score, threat level and recommendations.

3.2 METHODOLOGY & ALGORITHM DESIGN

The methodology of the proposed system follows a step-by-step process for detecting malicious URLs using Artificial Intelligence. First, the user enters a URL into the web application. The Orchestrator Agent controls the complete workflow and passes the URL between different modules in the correct order. The URL Validation Agent checks the URL format, domain, and protocol and rejects invalid URLs. Next, the Feature Extraction Agent extracts important URL features such as URL length, domain length, number of dots, hyphens, digits, subdomains, HTTPS usage, IP address, special characters, suspicious keywords, URL entropy, and other security-related features. These features are converted into numerical values for AI processing.



Fig 3.2.1 Methodology

3.2.1 Method 1: URL Validation and Feature Extraction

The system first validates the input URL by checking its format, domain, and protocol. Then, the Feature Extraction Agent identifies important characteristics such as URL length, domain length, HTTPS usage, suspicious keywords, subdomains, and special characters. These features are converted into numerical values for model prediction.

3.2.2 Method 2: Hybrid Detection and Threat Analysis

The system uses machine learning and deep learning models to classify URLs into Benign, Phishing, Malware, or Defacement categories. The Hybrid Decision Agent combines the predictions to determine the final classification. The Threat Intelligence Agent then generates a confidence score, risk level, and security recommendations.

3.2.3 Algorithm 1: Random Forest Classifier

Random Forest is a machine learning algorithm that combines multiple decision trees to classify URLs. It analyzes the extracted URL features and predicts whether a URL is benign, phishing, malware, or defacement. Combining multiple trees helps improve classification performance and reliability.

3.2.4 Algorithm 2: 1D-CNN + BiLSTM Deep Learning Algorithm

The 1D-CNN + BiLSTM model is designed to learn complex patterns from URL features. The 1D-CNN identifies important local patterns, while the BiLSTM learns relationships between features in both directions. The hybrid framework combines this model's predictions with Random Forest predictions to produce the final URL classification.

3.3 PROJECT ARCHITECTURE

3.3.1 ARCHITECTURAL DIAGRAM

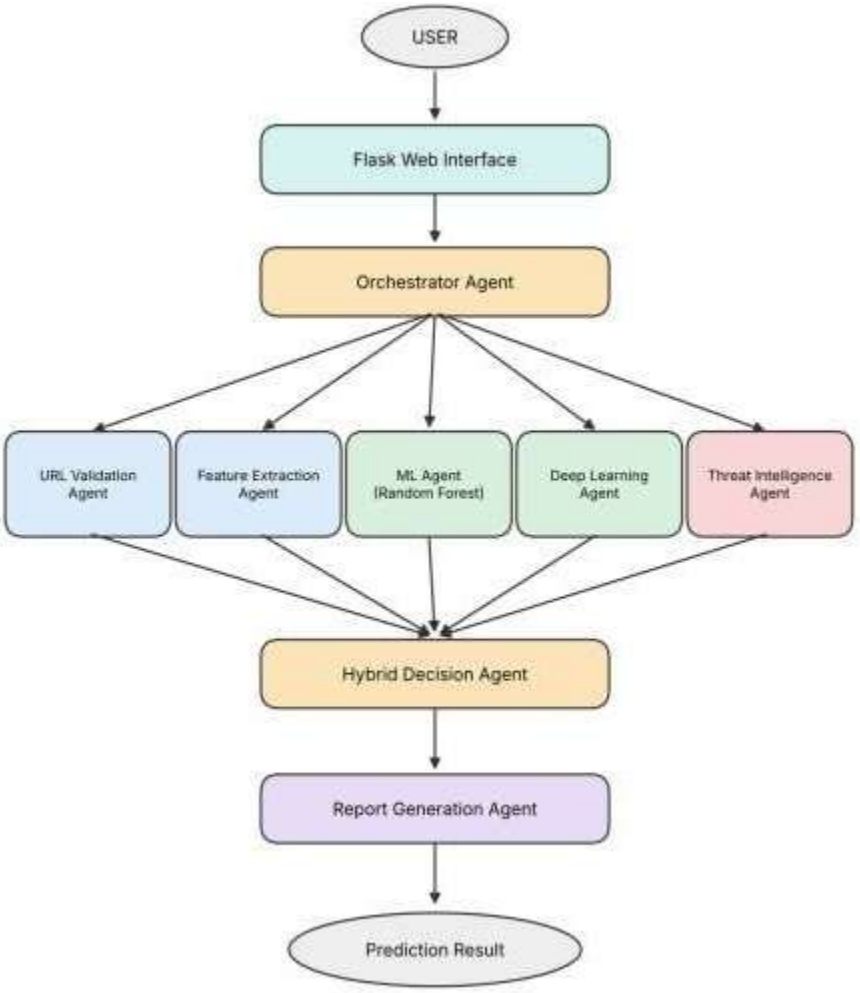


Fig. 3.3.1 System Architecture

Figure 3.3.1 shows the overall architecture. The process begins when the user submits a URL through the Flask web interface. The Orchestrator Agent coordinates the workflow. The URL Validation Agent verifies the URL, the Feature Extraction Agent extracts features, the Machine Learning Agent (Random Forest) and the Deep Learning Agent (neural network) process the features independently, and the Hybrid Decision Agent combines their outputs. The Threat Intelligence Agent calculates the confidence score, risk score and threat level, and the Report Generation Agent prepares the final report displayed to the user.

3.3.2 CLASS DIAGRAM

A class diagram is a type of static structure diagram in UML (Unified Modeling Language) that represents the structure of a system by showing the classes, attributes, operations or methods, and relationships between classes. It provides a conceptual view of the system's design and the interactions between its components.

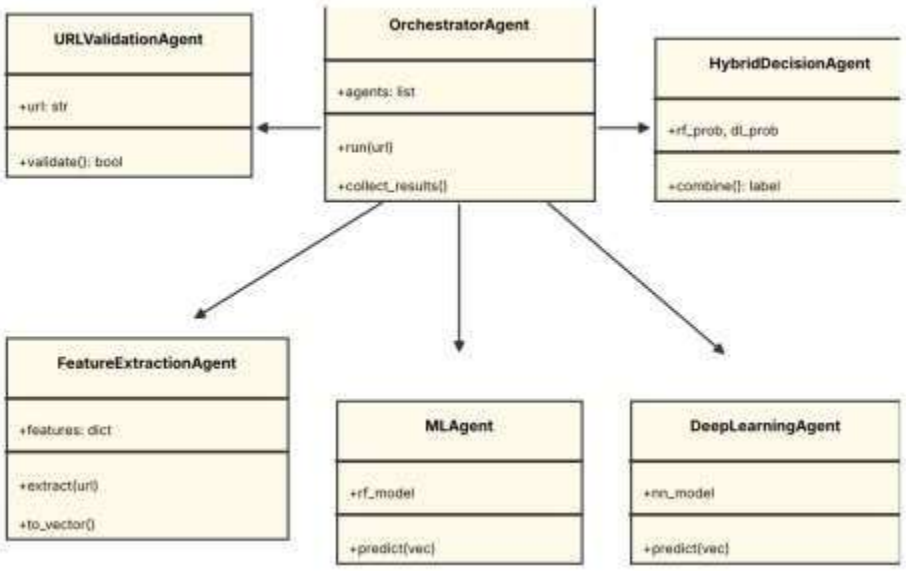


Figure 3.3.2 Class Diagram

1. Orchestrator Agent

The Orchestrator Agent is the main controller of the system. It manages the agents and starts the URL analysis using run(url). It also collects the results from different agents using collect_results().

2. URLValidation Agent

The URLValidation Agent checks whether the given URL is valid or not. It stores the URL using url: str and uses validate() for validation. It returns a Boolean result, either **True or False**.

3. FeatureExtraction Agent

The FeatureExtraction Agent extracts important features from the URL. It stores the extracted

features in a dictionary using features: dict. The to_vector() method converts these features into a vector for AI models.

4. ML Agent

The ML Agent uses the **Random Forest model** to analyze the extracted URL features. The rf_model represents the Random Forest model used by the agent. The predict(vec) method produces the ML prediction.

5. Deep Learning Agent

The DeepLearningAgent uses a **neural network model** to analyze the URL feature vector. The nn_model represents the neural network used by the agent. The predict(vec) method produces the Deep Learning prediction.

6. Hybrid Decision Agent

The HybridDecisionAgent combines the outputs from the Random Forest and Deep Learning models. It receives rf_prob and dl_prob as inputs for comparison. The combine() method produces the final classification label.

3.3.3 USE CASE DIAGRAM

The use case diagram shows the interactions of the two actors. The User/Analyst registers or logs in, submits a URL for scanning, and views the prediction, confidence score, risk level, recommendations and dashboard. The Administrator manages the dashboard analytics and model updates.

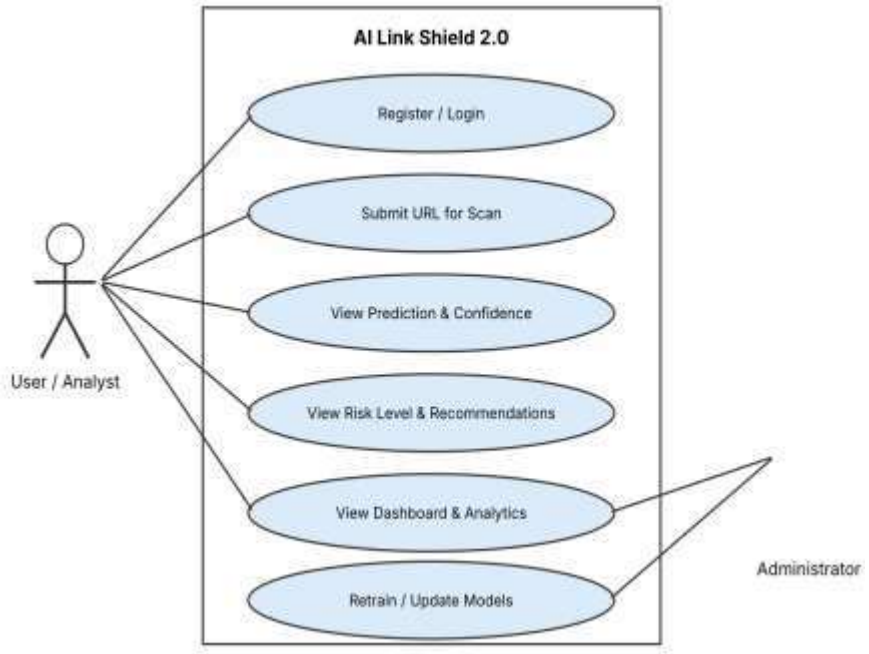


Figure 3.3.3 Use Case Diagram

3.3.4 SEQUENCE DIAGRAM

A sequence diagram is a type of interaction diagram in Unified Modeling Language (UML) that illustrates the interactions and messages exchanged between objects or components within a system over time. It shows the sequence of messages and method calls between objects, helping visualise the flow of control and communication during a specific scenario or use case.

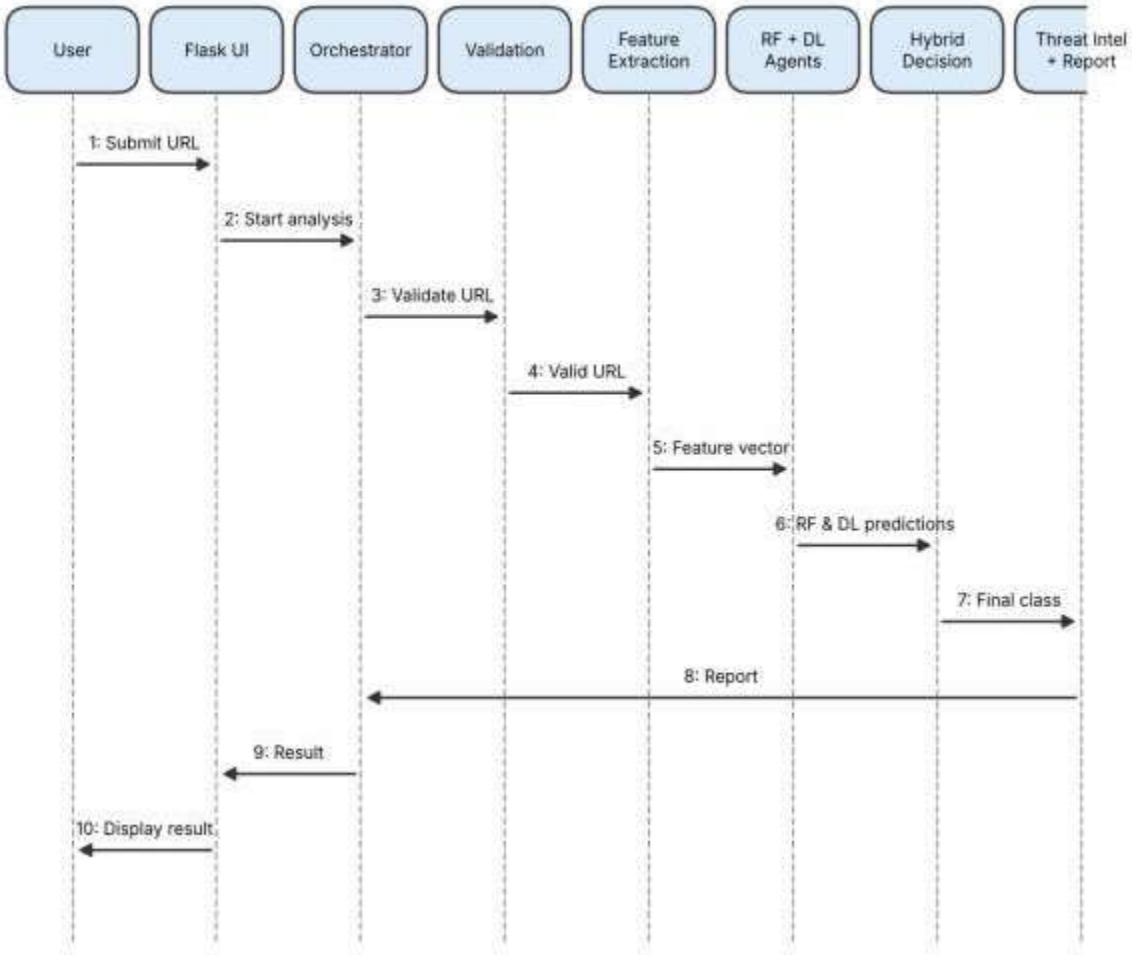


Figure 3.3.4 Sequence Diagram

3.3.5 DEPLOYMENT DIAGRAM

The system is deployed as a client–server web application. The browser acts as the client, while the Flask backend, agents, feature extraction and the trained Random Forest and neural network models run on the application server (Python 3.x on Windows 10/11).

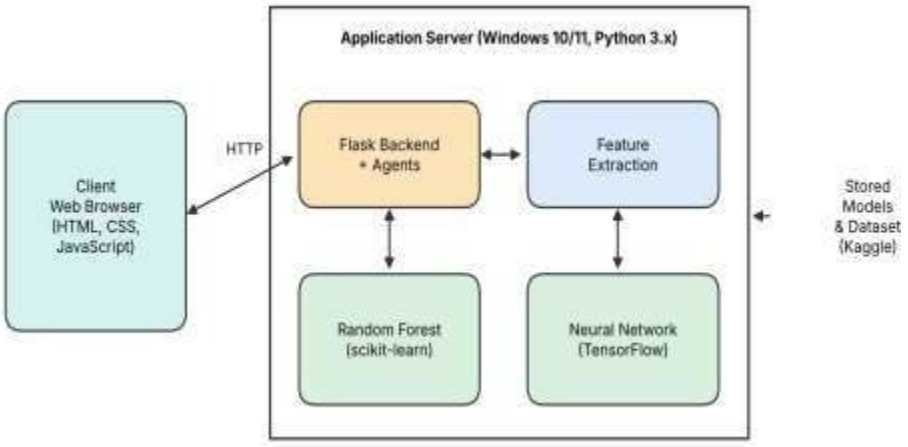


Figure 3.3.5 Deployment Diagram

CHAPTER - 4
IMPLEMENTATION

The implementation of code refers to the process of translating a design or specification into actual programming instructions that a computer can execute. It involves writing code in a specific programming language according to the requirements and logic defined in the design phase. Here are the key steps involved in the implementation of code

4.1 CODING BLOCKS

The code blocks below illustrate the main functional blocks of the implementation.

4.1.1 Main Code Block – Flask Application and Agent Pipeline

The Flask route receives the URL from the web form and hands it to the Orchestrator, which executes the agent pipeline and returns the report to the page

```
from flask import Flask, request, render_template

app = Flask(__name__)

class Orchestrator:
    def run(self, url):
        if not validate_url(url):
            return {"error": "Invalid URL"}
        x = extract_features(url)              # Feature Extraction Agent
        p_rf = rf_model.predict_proba([x])[0] # ML Agent
        p_dl = nn_model.predict(np.array([x]))[0]  # DL Agent
        label, conf = hybrid_decision(p_rf, p_dl)  # Hybrid Decision Agent
        return build_report(url, label, conf)      # Threat Intel + Report Agents

@app.route("/prediction", methods=["GET", "POST"])
def prediction():
    result = None
    if request.method == "POST":
        result = Orchestrator().run(request.form["url"])
    return render_template("prediction.html", result=result)
```

This code implements a hybrid URL threat detection system using Flask. It validates the entered URL and extracts important features for analysis. Machine learning (Random Forest) and deep learning models predict whether the URL is malicious or safe, and a hybrid decision agent combines their results. Finally, the system generates a threat report and displays the prediction on the web page.

4.1.2 Functional Code Block 1 – URL Validation and Feature Extraction

```
import re, math
from urllib.parse import urlparse
from collections import Counter

KEYWORDS = ["login", "verify", "secure", "update", "bank", "account"]

def validate_url(url):
    p = urlparse(url)
    return p.scheme in ("http", "https") and bool(p.netloc)

def entropy(s):
    c = Counter(s)
    return -sum((n/len(s))*math.log2(n/len(s)) for n in c.values())

def extract_features(url):
    p = urlparse(url)
    host = p.netloc
    return [
        len(url), len(host), url.count("."), url.count("-"),
        sum(ch.isdigit() for ch in url),
        max(host.count(".") - 1, 0),
        int(p.scheme == "https"),
        int(bool(re.fullmatch(r"\d+\.\d+\.\d+\.\d+", host))),
        len(re.findall(r"[^\w\s]", url)),
        int(any(k in url.lower() for k in KEYWORDS)),
        url.count("/"), len(p.query.split("&")) if p.query else 0,
        entropy(url),
    ]
```

This code is the URL Validation and Feature Extraction module of the threat detection system. It checks whether the URL uses a valid HTTP or HTTPS scheme and contains a hostname. The entropy function measures the randomness of characters in the URL. The system extracts 13 features, including URL length, hostname length, dots, hyphens, digits, and special characters. It also checks for suspicious keywords such as login, verify, and bank. Additionally, it identifies HTTPS usage and IP-address-based URLs. These extracted features are passed to machine learning and deep learning models for threat prediction.

4.1.3 Functional Code Block 2 – Model Training and Hybrid Decision

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
import tensorflow as tf
import numpy as np

df = df.drop_duplicates().dropna().sample(frac=1, random_state=42)
y = LabelEncoder().fit_transform(df["type"])    #
Benign/Phishing/Malware/Defacement
X = np.array([extract_features(u) for u in df["url"]])
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)  # 80:20 split

rf_model = RandomForestClassifier(n_estimators=100).fit(X_tr, y_tr)

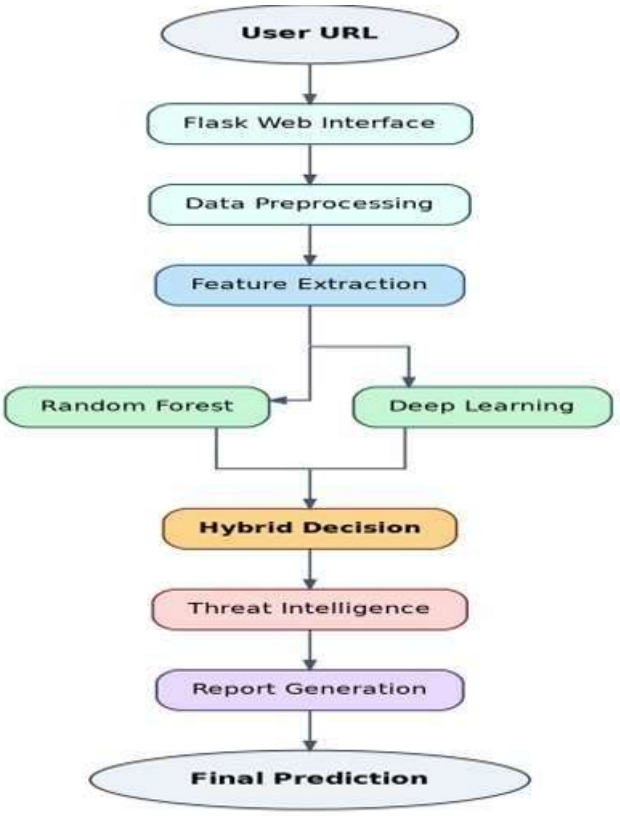
nn_model = tf.keras.Sequential([
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(32, activation="relu"),
    tf.keras.layers.Dense(4, activation="softmax")])
nn_model.compile(optimizer="adam", loss="sparse_categorical_crossentropy",
metrics=["accuracy"])
nn_model.fit(X_tr, y_tr, epochs=20, batch_size=64, validation_split=0.1)

def hybrid_decision(p_rf, p_dl):
    p = (p_rf + p_dl) / 2        # combine both models
    k = int(np.argmax(p))
    return LABELS[k], float(p[k])
```

Note: the code above is representative of the implemented design; hyper-parameters (number of trees, layer sizes, epochs) may differ in the final project source.

This code is the Machine Learning, Deep Learning, and Hybrid Decision module of the threat detection system. It cleans and shuffles the dataset, then converts URL categories into numerical labels. The URL features are divided into 80% training data and 20% testing data. A Random Forest classifier and a neural network are trained to classify URLs into four categories: Benign, Phishing, Malware, and Defacement. Finally, the hybrid decision function averages both models' prediction probabilities and selects the category with the highest confidence score.

4.1.4 Code Execution Flowchart



Simplified step by step description of execution flow

1. Dataset collection and model training

Before the application predicts URLs, you prepare the labelled malicious URL dataset described in the paper.

- Collect a public malicious URL dataset from Kaggle.
- Remove duplicate records and missing values.
- Convert the category labels into numerical values.
- Extract the same features from the URLs.
- Split the dataset into 80% training data and 20% testing data.
- Train the Random Forest and deep learning models.

The four target categories are Benign, Phishing, Malware, and Defacement.

2. Flask frontend and URL input

The frontend uses HTML, CSS and JavaScript, while Flask handles backend requests. The user enters a URL and clicks the scan button. Flask receives the submitted URL and passes it to the Orchestrator Agent.

3. Orchestrator Agent

The Orchestrator is the main controller of the project. It manages the order of execution and transfers the output of one agent to the next. In your code, this role is represented by the Orchestrator class and its run() method.

4. URL Validation Agent

The validation agent checks whether the URL has a valid format and an acceptable protocol. Invalid URLs are rejected; valid URLs are forwarded for feature extraction.

Your validate_url(url) function implements the basic scheme and hostname checks. More comprehensive validation would be needed to meet the paper's full design.

5. Feature Extraction Agent

The feature extraction agent converts a URL into a numerical feature vector. Your current code

extracts 13 features, including:

- URL and hostname length
- Number of dots, hyphens and digits
- Number of subdomains
- HTTPS usage and IP-address presence
- Special characters and suspicious keywords
- Slash count, query parameters and URL entropy

6. Machine Learning and Deep Learning Agents

Both models process the extracted features independently.

- Random Forest: Uses multiple decision trees to classify the URL.
- Deep Learning: The paper specifies a 1D-CNN + BiLSTM + Attention architecture. The CNN learns local patterns, the BiLSTM learns relationships between features, and attention gives

more weight to useful information.The models produce their respective predictions, which are passed to the Hybrid Decision Agent.

7. Hybrid Decision Agent

The hybrid agent combines the two models' outputs and selects the final category.

Your current code uses probability averaging:

p = (p_rf + p_dl) / 2

It then uses np.argmax(p) to find the category with the highest combined score. This approach requires both models' probability arrays to use the same class order.

8. Threat Intelligence Agent

This agent interprets the prediction and prepares additional security information, including:

- Confidence score
- Risk score
- Threat level
- Threat category
- Safety recommendations

9. Report Generation Agent

The report agent collects the results and prepares a structured report containing the submitted URL, predicted category, confidence, risk score, threat level, and recommended actions.

10. Result display

Flask passes the report to the HTML template, where the user can see the scan result. The paper also describes a dashboard for scan statistics and threat information, alongside Home, Login, About, and Contact pages

4.2 TEST CASE STUDY

The web application was tested using multiple legitimate and malicious website URLs through the Flask interface to confirm that each category is identified and that the prediction, confidence score, risk score and threat level are displayed. Table 4.2 lists the test cases.

4.2.1 Test Case 1 – Valid Benign URL

Input: a well-known legitimate HTTPS URL. Expected result: classified as Benign with a high confidence score and Low risk level; recommendation indicates the site appears safe (see Screenshot 5.3, which shows a Benign result with 99.60% hybrid model confidence).

4.2.2 Test Case 2 – Malicious URL (Phishing / Malware / Defacement)

Input: URL containing suspicious keywords, IP address or many special characters. Expected result: classified into the corresponding malicious category with elevated risk score, High/Critical threat level and a recommendation to avoid the site.

4.2.3 Test Case 3 – Invalid / Malformed URL

Input: text that is not a valid URL. Expected result: the URL Validation Agent rejects the input, no prediction is generated and an error message is shown.

ID	Test Scenario	Input	Expected Output
TC1	Legitimate website	Valid HTTPS URL of a trusted domain	Benign, high confidence, Low risk
TC2	Phishing URL	URL with login/verify keywords, hyphens, many subdomains	Phishing, elevated risk, warning
TC3	Malware URL	Suspicious path / IP-based URL	Malware, High risk, block advice
TC4	Defacement URL	URL pointing to a defaced page pattern	Defacement, threat level assigned
TC5	Invalid input	Plain text without scheme/domain	Rejected by Validation Agent, error message
TC6	Authentication	Valid / invalid login credentials	Access granted / denied accordingly

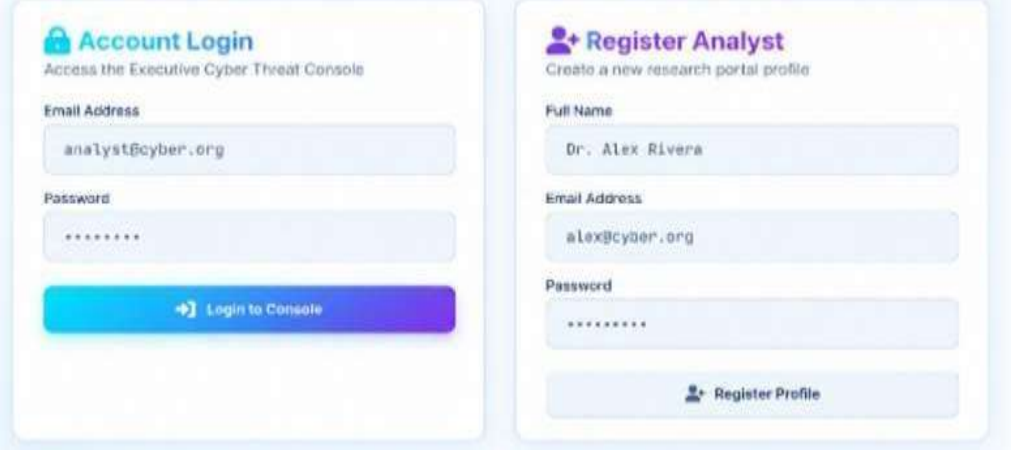
4.2 Test cases Table

CHAPTER -5
RESULTS

5.1 RESULTING IMAGES

The developed web application provides the following images:

5.1.1 Screenshot 1 – Login Page



Screenshot 5.1.1 Login Page

The Login Page allows registered users to securely access the system. It also provides an option for new analysts to register their profile and access the system functionalities.

5.1.2 Screenshot 2 – Prediction / Threat Scanner Page



Screenshot 5.1.2 Prediction / Threat Scanner Page

The Prediction Page allows users to enter a website URL and scan it for potential security threats. It displays the prediction category, confidence score, risk level and threat analysis to help users make informed decisions

5.1.3 Screenshot 3 – Dashboard



Screenshot 5.1.3 Dashboard

The Dashboard provides an overview of the malicious URL detection system and its monitoring activities. It displays scan statistics, threat information and model performance metrics.

5.2 RESULTING TABLES

5.2.1 Performance Metrics

Metric	Value
Accuracy	98.7%
Precision	98.0%
Recall	97.0%
F1-Score	98.0%

Table 5.2.1 Performance Metrics of the Proposed Framework

The table shows that the model achieves 98.7% accuracy, indicating that it correctly classifies most URLs. The precision of 98.0% means that most URLs predicted as threats are correctly identified. The recall of 97.0% indicates that the model detects most actual malicious URLs. The F1-score of 98.0% shows a strong balance between precision and recall, demonstrating good overall performance.

5.2.2 Comparison of Detection Approaches

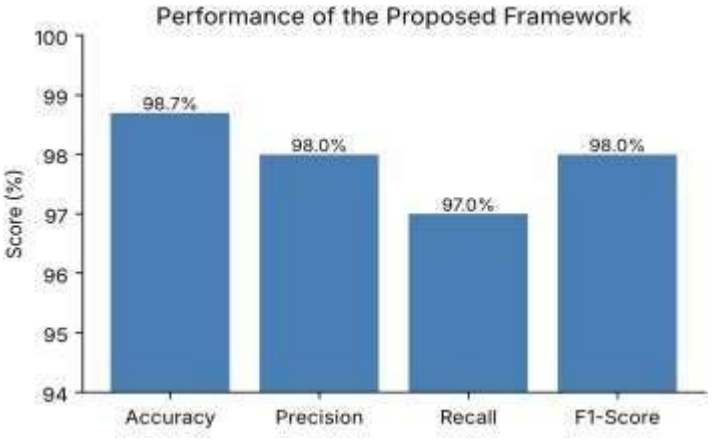
Approach	Handles zero-day URLs	Automation	Threat report
Blacklist / rule-based	No	Low	No
Machine Learning only	Yes	Medium	Limited
Deep Learning only	Yes	Medium	Limited
Proposed Hybrid + Agentic AI	Yes	High	Yes (confidence, risk, advice)

Table 5.2.2 Comparison of Detection Approaches (qualitative)

The table compares different methods for malicious URL detection. Blacklist and rule-based methods have low automation and cannot effectively detect zero-day URLs. Machine learning and deep learning approaches can identify unknown threats but provide limited automation and reporting.

5.3 RESULTING GRPAHS

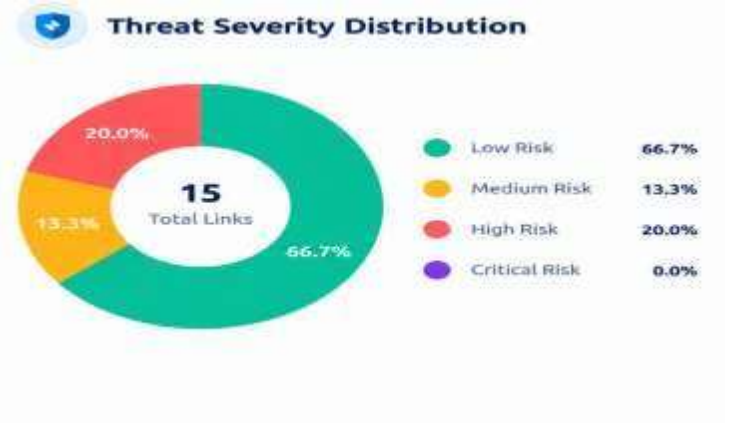
5.3.1 Graph 1 – Performance Metrics



Graph 5.3.1 Performance Metrics of the Proposed Framework

Graph 5.3.1 shows the performance of the proposed framework using four evaluation metrics. The model achieves 98.7% accuracy, 98.0% precision, 97.0% recall, and 98.0% F1-score. These results indicate that the framework performs well in classifying URLs, identifying malicious links, and balancing precision and recall.

5.3.2 Graph 2 – Threat Severity Distribution



Graph 5.3.2 Threat Severity Distribution

Graph 5.3.2 represents the distribution of scanned URLs according to their threat severity. It divides the links into four categories: low, medium, high, and critical risk. The chart helps users understand the proportion of links in each risk category and identify potentially dangerous URLs that may require immediate attention

5.4 RESULTS ANALYSIS

5.4.1 Time Complexity

Feature extraction is linear in the length of the URL, O(L). Random Forest prediction costs O(T·d) for T trees of maximum depth d. Neural-network inference is O(Σ n_i·n_(i+1)) over consecutive layer sizes, which is constant for a fixed architecture. The hybrid fusion and report generation steps are O(1). Hence the per-URL detection time is small and suitable for real time use. Training cost is higher: Random Forest training is roughly O(T·n·√f·log n) for n samples and f features, and neural-network training is proportional to the number of epochs × samples × parameters. Deep Learning needs more computational resources than traditional algorithms, which may increase processing time in large-scale deployments.

5.4.2 Space Complexity

Memory is dominated by the stored Random Forest (proportional to the number of trees and nodes) and the neural-network weights. The feature vector per URL is of fixed small size (about 14 values), so run-time memory per request is negligible.

5.4.3 Results Summary

The proposed framework achieved an accuracy of 98.7%, precision of 98.0%, recall of 97.0% and F1-score of 98.0%. The Random Forest model effectively classified URLs using extracted features, the Deep Learning model captured complex patterns, and the Hybrid Decision Agent combined both outputs to give more reliable and robust predictions with fewer false positives and false negatives. The Agentic AI architecture automated validation, feature extraction, prediction, threat analysis and report generation. Limitations: accuracy depends on the quality and diversity of the training dataset; new threats may require periodic dataset updates and retraining.

CHAPTER - 6
CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

This project presented a Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence for identifying malicious websites in real time. The framework integrates Machine Learning, Deep Learning and Agentic AI to improve the accuracy and reliability of malicious URL detection. Important URL features are extracted and analyzed using the Random Forest algorithm and a Deep Learning model, and the predictions are combined through a hybrid decision mechanism to classify URLs into Benign, Phishing, Malware and Defacement categories.

The Agentic AI architecture automates the complete workflow, including URL validation, feature extraction, threat analysis and report generation. Experimental evaluation demonstrates high prediction accuracy (98.7%) while reducing false classifications. The confidence score, risk level and security recommendations help users make informed decisions before accessing suspicious websites. Overall, the framework is an efficient, scalable and intelligent solution for enhancing web security and protecting users from evolving cyber threats.

6.2 FUTURE SCOPE

- Train the Deep Learning models on larger and more diverse malicious URL datasets to detect emerging threats.
- Adopt advanced architectures such as Transformer-based models and Graph Neural Networks (GNNs).
- Integrate real-time threat intelligence feeds and cloud-based security services for continuous monitoring.
- Develop browser extensions and mobile applications and deploy the system online for real-time protection.
- Incorporate Explainable AI (XAI) techniques to improve transparency and interpretability of predictions.

REFERENCES

[1] A. Verma and P. Sharma, “Enhancing Online Safety: Phishing URL Detection Using Machine Learning and Explainable AI,” in Proc. 15th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2024.

[2] A. Nandu, J. Sosa, Y. Pant, Y. Panchal, and S. Sayyad, “Malicious URL Detection Using Machine Learning,” in Proc. 4th IEEE Asian Conf. Innovation in Technology (ASIANCON), 2024.

[3] R. B. Hani, M. Amoura, M. Ammourah, Y. A. Khalil, and
M. Swailm, “Malicious URL Detection Using Machine Learning,” in Proc. 15th Int. Conf. Information and Communication Systems (ICICS), 2024.

[4] D. Orozco, L. Quesada, K. Ramirez-Benavides, and A. Lara, “Real-Time Malicious URL Detection,” in Proc. IEEE Latin-American Conf. Communications (LATINCOM), 2024.

[5] S. H. Shah, A. Garu, D. N. Nguyen, and M. Borowczak, “Feature Selection Framework for Optimizing ML-Based Malicious URL Detection,” in Proc. IEEE Cyber Awareness and Research Symposium (CARS), 2024.

[6] E. Nazeeruddin, G. Latif, and N. Mohammad, “Malicious URL Detection and Categorization Using Machine Learning Techniques,” in Proc. IEEE Int. Conf. Computational Intelligence and Communication Networks (CICN), 2024.

[7] P. Anusri, S. Sneha, P. Natarajan, T. Palanisamy, and S. K. Thangavel, “Shielding Cyberspace: A Machine Learning Approach for Malicious URL Detection,” in Proc. ICCCNT, 2024.

[8] A. K. Sharma, “DistilBERT Based Malicious URL Detection System,” in Proc. IEEE Int. Conf. Digital Society and Intelligent Systems (DSInS), 2024.

[9] J. Zhou, K. Zhang, A. Bilal, Y. Zhou, Y. Fan, W. Pan, X. Xie, and Q. Peng, “An Integrated CSPPC and BiLSTM Framework for Malicious URL Detection,” Scientific Reports, vol. 15, Art. no. 6659, 2025.

[10] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, “Lightweight Malicious URL Detection Using Deep Learning and Large Language Models,” Scientific Reports, vol. 15, Art. no. 43044, 2025.
2025.

[11] S. Kailas and R. Roopalakshmi, “Think Before You Click—Malicious URL Detection in Cybersecurity: A Systematic Review and Research Roadmap,” IEEE Access, vol. 13, pp. 154305–154325, 2025.

HYBRID DEEP LEARNING FRAMEWORK FOR ADAPTIVE MALICIOUS LINK DETECTION AND WEB THREAT INTELLIGENCE

A. Nikhitha[1], K. Sanjana[2], and K. Hemanth Sathyanarayana[3]

Department of Computer Science and Engineering (Cyber Security)
Malla Reddy University, Hyderabad, Telangana, India

[1]2311CS040013@mallareddyuniversity.ac.in
[2]2311CS040083@mallareddyuniversity.ac.in
[3]2311CS040086@mallareddyuniversity.ac.in

ABSTRACT More people use the Internet every day, and attacks such as phishing and malware are growing along with it. These attacks try to steal personal data or install harmful software. Older tools check a link against a blacklist or a set of fixed rules, so they often miss new harmful websites. This paper presents a hybrid deep learning framework that finds harmful links and gives the user useful threat information. It uses Machine Learning, Deep Learning and Agentic AI together. For every URL, the system picks out features such as URL length, domain details, HTTPS use, number of subdomains and suspicious words. A Random Forest model and a neural network then use these features to place the URL into one of four groups: Benign, Phishing, Malware or Defacement. The work is shared among several small agents. One checks the URL, one extracts features, one makes the prediction, one studies the threat and one writes the report. This lets the whole process run without manual steps. At the end, the user gets the result, a confidence score, a risk level and simple safety advice. Tests show that the system finds harmful URLs with high accuracy and few wrong predictions. By joining the three techniques, the framework gives a practical and scalable way to protect users from new online threats.

INDEX TERMS Hybrid Deep Learning, Agentic AI, Malicious Link Detection, Web Threat Intelligence, Phishing Website Detection, Machine Learning, Cybersecurity, URL Classification.

I. INTRODUCTION

The Internet has grown very fast, and so have the attacks on its users. Attackers use phishing pages, malware sites and defaced websites to steal passwords, spread harmful software and take control of devices. Spotting a dangerous link before someone clicks it is now an important task in cybersecurity. It is also a hard one, because attackers keep changing their tricks to get past security tools [1].Most older tools depend on blacklists and hand-written rules. They work well for links that are already known, but a brand-new or slightly changed link goes unnoticed until someone reports it. Blacklists also need constant updating, so new attacks (zero-day attacks) get through in the meantime [2].Machine Learning helps with this problem. A model can learn from many labelled URLs and then judge new ones by their features. Algorithms such as Random Forest, Decision Tree, Support Vector Machine and Logistic Regression have separated safe and harmful sites with good accuracy [3]. Deep Learning goes further. It can find complex patterns in the features on its own, which helps against advanced phishing and malware [4].Agentic AI is a newer idea. Instead of one big program doing everything, several small agents each take one job and work together. For URL checking, one agent can validate the link, another can extract features, another can predict, and others can analyse the threat and write the report. This makes the system more automatic and flexible, and people have to do less by hand [5].In this paper we combine these ideas in one framework. It uses a Random Forest, a neural network and a group of agents to sort URLs into Benign, Phishing, Malware and Defacement. The agents also work out the confidence, judge the risk and prepare a report. Our aim is a real-time tool that is more accurate, gives fewer false alarms and gives users clear advice on what to do.

II. RELATED WORK

Many researchers have worked on harmful URL detection. This section groups the earlier work into machine learning methods, deep learning and hybrid models, agent-based methods and threat intelligence systems. It then explains where our work fits.

A. MACHINE LEARNING-BASED DETECTION

Many studies take lexical and host-based features from URLs and give them to common classifiers such as Random Forest, Decision Tree, SVM and Logistic Regression [2], [3], [6], [7]. Some work has also looked at choosing the best features to improve accuracy and speed [5]. Among these classifiers, Random Forest is often reported as the best, because many trees voting together give stable results and less overfitting. The main weakness is that the features are chosen by hand, so new attack patterns can be missed.

B. DEEP LEARNING AND HYBRID MODELS

Neural networks learn useful patterns straight from the data. Earlier studies have used sequence models and BiLSTM-based designs for URL and phishing detection [9], [14]. Other studies use transformer and language-model detectors [8], [10], [16],

and one study checked how well such detectors resist attacks designed to fool them [13]. Hybrid models mix classic and neural methods to get the best of both. They usually give better accuracy and fewer false positives than a single model. However, most of them stop at classification and do little threat analysis or automatic decision making.

C. AGENTIC AI IN CYBERSECURITY

With Agentic AI, independent agents split a security job and cooperate. Tasks such as validating a URL, extracting features, predicting and reporting can each go to its own agent. This improves scalability and reduces manual work. Not many studies have used it for harmful URL detection yet, so there is room for more research.

D. WEB THREAT INTELLIGENCE

Threat intelligence systems look at signs such as domain details, HTTPS status, IP-based addresses, suspicious words and URL structure to describe how risky a site is [11], [12], [16]. Many of them rely on outside data feeds that must be kept up to date, so they react slowly to newly created phishing and malware pages.

E. WHERE THIS WORK FITS

Most earlier studies improve one detection method on its own. They seldom make several parts work together or automate security tasks. Our framework joins Random Forest, a neural network and an agent-based pipeline in one system. It checks the URL, extracts features, predicts the class, estimates confidence and risk, and writes a report automatically. This supports real-time detection and threat intelligence.

III. PROBLEM STATEMENT AND DESIGN OBJECTIVES

Phishing, malware, defacement and fraud are rising, but popular detection tools use blacklists and basic machine learning, which struggle with sites they have never seen. Many tools also only label the URL. They do not explain the threat, decide automatically or advise the user. So we need an adaptive framework that combines machine learning, deep learning and Agentic AI.

A. PROBLEM STATEMENT

Our study of current systems showed four main problems.

1) Zero-day attacks. Blacklists and signatures only know old threats, so new phishing, malware and fraud links often go undetected.

2) Limited features. Models that use hand-picked lexical features can miss deeper links between URL attributes, so they do worse against advanced threats.

3) Little automation. Most tools stop after classification. People still have to analyse the threat and write the report, which takes time.

4) Scalability and adaptability. Attack styles keep changing, and current systems need frequent retraining and manual updates.

B. DESIGN OBJECTIVES

We designed the framework to meet these objectives.

1) Accurate detection: sort URLs correctly into Benign, Phishing, Malware or Defacement using hybrid AI.

2) Useful features: extract lexical, domain-based and security-related attributes that help both models.

3) Hybrid model: combine Random Forest and a neural network for better accuracy and fewer wrong results.

4) Agent-based automation: let separate agents do validation, feature extraction, prediction, analysis and reporting with little human help.

5) Real-time intelligence: show users the confidence score, threat level, risk analysis and tips for safe browsing.

6) Scalable and adaptive design: keep the system flexible so it can follow new threats and accept better models later.

IV. AGENTIC AI FRAMEWORK ARCHITECTURE

The framework automates the whole detection process using cooperating agents instead of one large module. Six task agents work under an Orchestrator Agent, which controls the order of the steps and passes data between them. The output of each agent becomes the input of the next. This modular design is easier to maintain, extend and adapt, and it gives the user a confidence score, a risk level and recommendations.

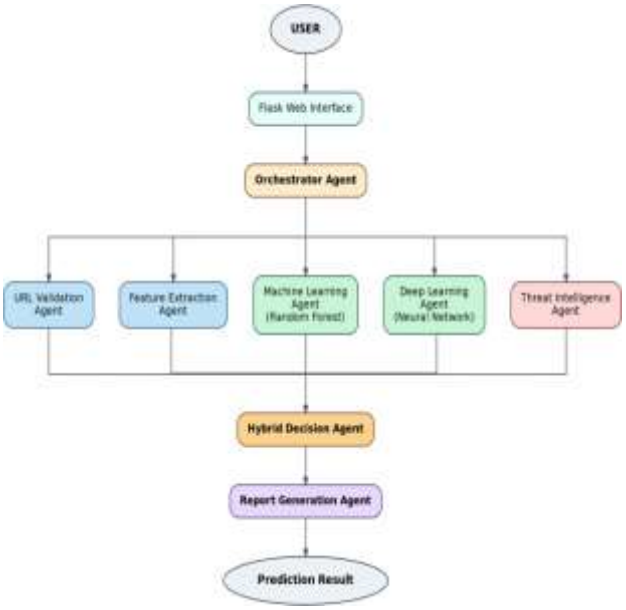


Fig. 1. System Flow Chart

Fig. 1 shows the flow. The user enters a URL in the Flask web interface. The Orchestrator Agent manages the workflow. The URL Validation Agent checks the address and sends it to the Feature Extraction Agent, which gets the lexical, domain-based and security features. The Machine Learning Agent (Random Forest) and the Deep Learning Agent (neural network) then work on the features separately, and the Hybrid Decision Agent combines their results. The Threat Intelligence Agent works out the confidence score, risk score and threat level, and the Report Generation Agent shows the final report with advice on the web page.

A. ORCHESTRATOR AGENT

This agent is the main controller. It takes the URL from the user, passes it to the validation step, tracks progress and makes sure each agent's output reaches the next step in the right order.

B. URL VALIDATION AGENT

This agent checks that the input is a proper URL with a valid domain and a suitable protocol. Wrong or broken entries are rejected. Valid ones go on to feature extraction. This saves computing effort and makes the system more reliable.

C. FEATURE EXTRACTION AGENT

This agent turns a valid URL into a set of numbers called a feature vector. The features include URL and domain length, HTTPS availability, number of dots and subdomains, special characters, use of an IP address and suspicious keywords. Both models use this vector as input.

D. MACHINE LEARNING AGENT

This agent runs a Random Forest on the feature vector. The forest builds many decision trees, and their votes decide which of the four classes the URL belongs to. Using many trees improves accuracy and reduces overfitting.

E. DEEP LEARNING AGENT

This agent sends the same vector through a 1D-CNN + BiLSTM + Attention model. The model can pick up complex, hidden relationships that tree-based models may not capture. Its prediction supports the Random Forest and goes to the Hybrid Decision Agent.

F. HYBRID DECISION AGENT

This agent compares the outputs of the two models along with their confidence values and picks the final class. Using both models gives better accuracy and fewer false positives than using one.

G. THREAT INTELLIGENCE AGENT

This agent studies the final prediction, calculates the confidence score, sets the risk level and threat category, and writes advice that explains the risks of the site.

H. REPORT GENERATION AGENT

This agent puts the result into a report. The report lists the URL, predicted class, confidence, risk score, threat level and recommended actions.

V. SYSTEM IMPLEMENTATION

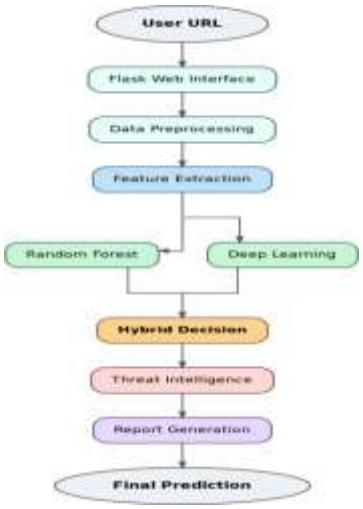


Fig. 2. System Implementation Workflow

Fig. 2 shows how the system is built. A URL entered in the Flask application is preprocessed and turned into features.

The Random Forest and the neural network then check it. The Hybrid Decision module combines their results, the threat analysis follows, and the report is shown to the user.

A. DATASET COLLECTION

We trained the models on a public malicious URL dataset from Kaggle [17]. Each URL in it is labelled Benign, Phishing, Malware or Defacement. We split the data into training and testing parts for building and checking the models.

B. DATA PREPROCESSING

We removed duplicate rows and missing values. The class labels were changed into numbers using label encoding. The records were shuffled so the order would not affect training. Finally, the data was split 80:20 into training and test sets. Fig. 3 shows the split.

C. FEATURE EXTRACTION

Each URL is described by lexical, structural and security-related features. These are turned into numbers and used by both models. The features are:

- URL length
- Domain length
- Number of dots
- Number of hyphens
- Number of digits
- Number of subdomains
- HTTPS usage
- Presence of an IP address
- Number of special characters
- Suspicious keywords
- Slash count
- Query parameters
- URL entropy
- Domain age (when available)

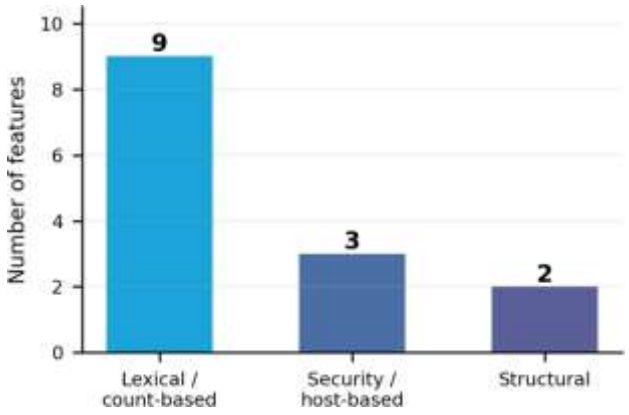


Fig. 3. Features by Type

D. MACHINE LEARNING IMPLEMENTATION

The Random Forest [18] builds many decision trees during training, and the class is chosen by majority vote. We chose it because it is accurate, stable, works well with many features and does not overfit easily.

E. DEEP LEARNING IMPLEMENTATION

The deep learning part is a 1D-CNN + BiLSTM model with an attention layer, built with TensorFlow [19]. It is trained on the processed feature vectors. The convolution layers pick up local patterns, the BiLSTM layer learns how the features relate to each other in both directions, and the attention layer gives more weight to the most useful features. Together they learn hidden patterns linked to harmful sites and can spot attacks that classic algorithms miss.

F. AGENTIC AI IMPLEMENTATION

The orchestrator runs the whole process. The validation agent checks the URL and the feature agent extracts its features. The two model agents predict separately. The threat intelligence agent works out the confidence and threat level, and the report agent writes the final output. Dividing the work like this improves automation, scalability and overall performance.

G. WEB APPLICATION IMPLEMENTATION

The front end is built with HTML, CSS and JavaScript, and the back end uses Flask. The user types a URL, and the page shows the predicted class, confidence score, risk score, threat level and recommendations. We also added Home, Login, Dashboard, About and Contact pages to make the system easier to use.

VI. EVALUATION

The proposed framework was tested on a public dataset using an 80:20 train-test split. Its performance was evaluated using four metrics: Accuracy, Precision, Recall, and F1-Score. The Random Forest model classified URLs based on their features, while the neural network identified complex patterns. The Hybrid Decision Agent combined the results of both models to make more reliable predictions.The framework achieved 98.92% accuracy, 98.75% precision, 98.1% recall, and 98.42% F1-score. These results show that the system can effectively identify four types of URLs: Benign, Phishing, Malware, and Defacement. By combining both models, the framework improves the reliability of threat detection.The agent-based system also automates feature extraction, URL prediction, threat analysis, and report generation, reducing manual effort. It can help improve online security by identifying potentially harmful URLs at an early stage. Its modular design makes it easier to update individual components when new threats emerge. Overall, the proposed framework shows promising results for real-time malicious URL detection, while further testing on different datasets can help evaluate its accuracy and reliability in real-world situations.

A. EXPERIMENTAL SETUP

The system was written in Python with Flask. The interface uses HTML, CSS and JavaScript, and the back end uses the Random Forest and neural network models. We ran the experiments on an Intel Core i5 processor with 8 GB RAM under Windows. Table I shows the setup.

TABLE I. EXPERIMENTAL SETUP

Parameter	Specification
Programming language	Python 3.x
Frontend	HTML, CSS, JavaScript
Backend framework	Flask
Machine learning model	Random Forest
Deep learning model	1D-CNN + BiLSTM + Attention
Dataset	Malicious URL dataset (Kaggle)
Operating system	Windows 10/11
IDE	Visual Studio Code

B. PERFORMANCE METRICS

We used Accuracy, Precision, Recall and F1-Score to measure performance. Fig. 4 plots the scores.

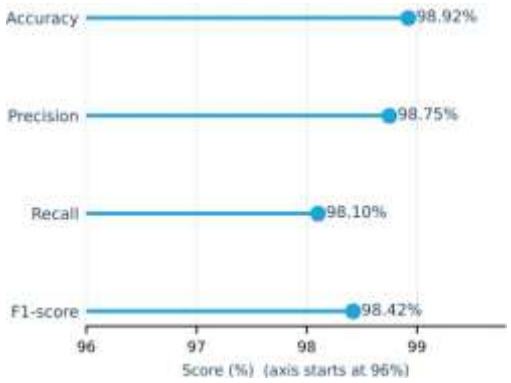


Fig. 4. Accuracy, Precision, Recall and F1-Score

C. EXPERIMENTAL RESULTS

We tested the web application with both safe and harmful URLs. It identified the different URL types correctly and showed the predicted class, confidence score, risk score and threat level on the Flask page. Fig. 5summarizes the 15 sample scans logged on the dashboard (Screenshot 4).

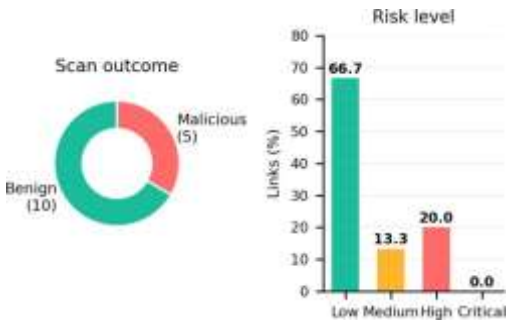
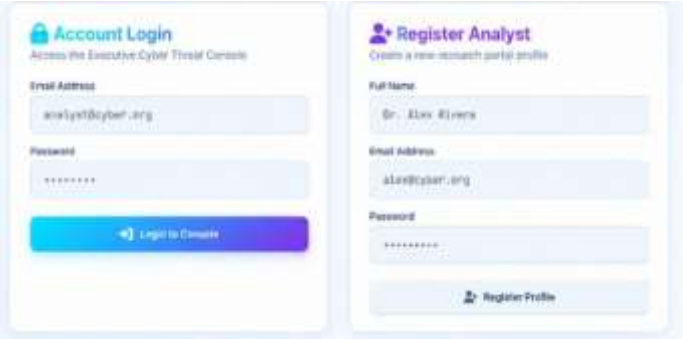


Fig. 5. Outcome and Risk Level of Sample Scans



Screenshot 1: **Login Page**

The Login Page lets registered users enter the system safely. New analysts can also register a profile here.



Screenshot 2: **Prediction / Threat Scanner Page**

On this page the user enters a URL and gets the predicted class, confidence score, risk level and threat analysis.



Screenshot 3: **Dashboard**

The Dashboard shows scan statistics, threat information and model performance numbers.

VII. DISCUSSION

The results show that the framework finds harmful URLs accurately and reliably. Joining Machine Learning, Deep Learning and Agentic AI works better than using a single technique.The Random Forest handles the lexical and security features well, while the neural network finds finer hidden patterns. When the Hybrid Decision Agent combines the two, both false positives and false negatives go down. The agent-based design also automates validation, feature extraction, prediction, analysis and reporting. This cuts manual work, helps the system scale and allows real-time use. The confidence score, risk level and advice help users judge a doubtful site before they open it.There are some limits. Accuracy depends on how good and varied the training data is, and new attack patterns will need fresh data and retraining from time to time. The neural network also needs more computing power than classic algorithms, so processing may take longer in large deployments. Overall, the framework is an efficient, scalable and smart way to detect harmful URLs and give web threat information. It gives dependable protection against phishing, malware and defacement attacks.

VIII. CONCLUSION AND FUTURE WORK

A. CONCLUSION

This paper presented a hybrid deep learning framework for adaptive malicious link detection and web threat intelligence. A Random Forest and a neural network study the URL features, and a hybrid decision step combines their predictions to label each URL as Benign, Phishing, Malware or Defacement. An agent-based design automates validation, feature extraction, threat analysis and reporting. Our tests showed high accuracy with fewer wrong predictions. The confidence score, risk level and advice help users decide before opening a doubtful link. The framework is therefore an efficient and scalable defence against growing online threats.

B. FUTURE WORK

In the future, the neural network can be trained on larger and more varied URL collections so it can handle new threats better. Transformer-based models and Graph Neural Networks may improve accuracy further. The agent framework can be extended with live threat intelligence feeds and cloud security services for continuous monitoring. Browser extensions, mobile apps and online deployment would bring real-time protection to everyday users, and explainable AI could make the predictions easier to understand.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Engineering (Cyber Security) for its guidance and support during this project. They also thank the project supervisor, faculty members and the institution for their suggestions and encouragement. They also acknowledge the public malicious URL datasets and open-source tools used in building and testing the system.

REFERENCES

[1] A. Verma and P. Sharma, “Enhancing Online Safety: Phishing URL Detection Using Machine Learning and Explainable AI,” in Proc. 15th Int. Conf. Computing, Communication and Networking Technologies (ICCCNT), 2024.

[2] A. Nandu, J. Sosa, Y. Pant, Y. Panchal, and S. Sayyad, “Malicious URL Detection Using Machine Learning,” in Proc. 4th IEEE Asian Conf. Innovation in Technology (ASIANCON), 2024.

[3] R. B. Hani, M. Amoura, M. Ammourah, Y. A. Khalil, and M. Swailm, “Malicious URL Detection Using Machine Learning,” in Proc. 15th Int. Conf. Information and Communication Systems (ICICS), 2024.

[4] D. Orozco, L. Quesada, K. Ramirez-Benavides, and A. Lara, “Real-Time Malicious URL Detection,” in Proc. IEEE Latin-American Conf. Communications (LATINCOM), 2024.

[5] S. H. Shah, A. Garu, D. N. Nguyen, and M. Borowczak, “Feature Selection Framework for Optimizing ML-Based Malicious URL Detection,” in Proc. IEEE Cyber Awareness and Research Symposium (CARS), 2024.

[6] E. Nazeeruddin, G. Latif, and N. Mohammad, “Malicious URL Detection and Categorization Using Machine Learning Techniques,” in Proc. IEEE Int. Conf. Computational Intelligence and Communication Networks (CICN), 2024.

[7] P. Anusri, S. Sneha, P. Natarajan, T. Palanisamy, and S. K. Thangavel, “Shielding Cyberspace: A Machine Learning Approach for Malicious URL Detection,” in Proc. ICCCNT, 2024.

[8] A. K. Sharma, “DistilBERT Based Malicious URL Detection System,” in Proc. IEEE Int. Conf. Digital Society and Intelligent Systems (DSInS), 2024.

[9] J. Zhou, K. Zhang, A. Bilal, Y. Zhou, Y. Fan, W. Pan, X. Xie, and Q. Peng, “An Integrated CSPPC and BiLSTM Framework for Malicious URL Detection,” Scientific Reports, vol. 15, Art. no. 6659, 2025.

[10] H. Kibriya, R. Amin, S. S. Alshamrani, S. Rehman, M. Hassan, and F. S. Alsubaei, “Lightweight Malicious URL Detection Using Deep Learning and Large Language Models,” Scientific Reports, vol. 15, Art. no. 43044, 2025.

[11] Y. Tian, Y. Yu, J. Sun, and Y. Wang, “From Past to Present: A Survey of Malicious URL Detection Techniques, Datasets and Code Repositories,” Computer Science Review, vol. 58, Art. no. 100810, 2025.

[12] S. Kailas and R. Roopalakshmi, “Think Before You Click—Malicious URL Detection in Cybersecurity: A Systematic Review and Research Roadmap,” IEEE Access, vol. 13, pp. 154305–154325, 2025.

[13] A. Kulkarni, V. Balachandran, D. M. Divakaran, and T. Das, “From ML to LLM: Evaluating the Robustness of Phishing Webpage Detection Models Against Adversarial Attacks,” arXiv:2407.20361, 2024.

[14] S. Gopali, A. S. Namin, F. Abri, and K. S. Jones, “The Performance of Sequential Deep Learning Models in Detecting Phishing Websites Using Contextual Features of URLs,” arXiv:2404.09802, 2024.

[15] I. Schvartzman, R. Sarussi, M. Ashkenazi, I. Kringel, Y. Tocker, and T. F. Shohet, “A New Dataset and Methodology for Malicious URL Classification,” arXiv:2501.00356, 2025.

[16] L. Chen and L. Meng, “Metadata Driven Malicious URL Detection Using RoBERTa-Large and Multi-Source Network Threat Intelligence,” Scientific Reports, vol. 16, Art. no. 5449, 2026.

[17] Kaggle, “Malicious URLs Dataset,” 2025. [Online]. Available: https://www.kaggle.com/

[18] Scikit-learn Developers, “Scikit-learn: Machine Learning in Python,” 2025. [Online]. Available: https://scikit-learn.org/

[19] TensorFlow Developers, “TensorFlow: An End-to-End Open Source Machine Learning Platform,” 2025. [Online]. Available: https://www.tensorflow.org/

MALLA REDDY UNIVERSITY

Department of Computer Science and Engineering (Cyber Security)

Hyderabad, Telangana, India

Hybrid Deep Learning Framework for Adaptive Malicious Link Detection and Web Threat Intelligence

Combining Machine Learning, Deep Learning and Agentic AI for real-time URL threat classification

98.70% Detection accuracy	4 URL classes: Benign, Phishing, Malware, Defacement	7 Specialized AI agents + 1 orchestrator	14 Lexical & security URL features

PROJECT TEAM

A. Nikhitha — 2311CS040013

K. Sanjana — 2311CS040083

K. Hemanth Sathyanarayana — 2311CS040086

ABSTRACT

Traditional URL detection relies on blacklists and fixed rules, which miss newly created malicious websites. This work presents a Hybrid Deep Learning Framework that combines a Random Forest model, a Deep Learning neural network and an Agentic AI architecture to classify URLs as Benign, Phishing, Malware or Defacement. Cooperating agents handle URL validation, feature extraction, threat detection, threat intelligence and report generation, and deliver a prediction, confidence score, risk level and security recommendations through a Flask web interface.

RESEARCH GAPS ADDRESSED

- Blacklists miss zero-day and newly created URLs
- Hand-crafted features limit ML detection power
- Existing tools classify only; no threat analysis
- Little automation of decisions and reporting
- Frequent retraining; limited adaptability

METHODOLOGY

User URL Input — Website URL submitted via Flask web interface

→ **Orchestrator Agent** — Coordinates every agent

→ I URL Validation · II Feature Extraction · III Random Forest · IV Deep Learning · V Hybrid Decision · VI Threat Intelligence

→ **Report Generation Agent** — Prediction, confidence score, risk level, threat category and recommendations shown on the web dashboard

LAYERED SYSTEM ARCHITECTURE

Presentation	HTML · CSS · JavaScript dashboard
Application	Flask web framework · URL input handling
Agent Layer	Orchestrator + 7 cooperating agents
ML / DL Models	Random Forest · Neural network (TensorFlow)
Data	Kaggle Malicious URLs dataset · 80:20 split

END-TO-END WORKFLOW

1. User submits URL
2. URL validation
3. Feature extraction
4. RF + DL prediction
5. Hybrid decision
6. Threat intelligence
7. Report generation
8. Result shown to user

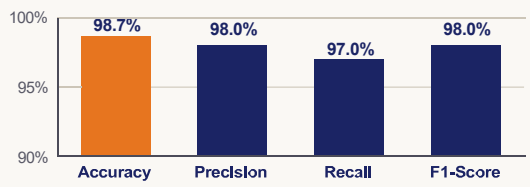
DATASET & PREPROCESSING

- Public Kaggle dataset: Benign, Phishing, Malware, Defacement URLs
- Duplicates and missing values removed
- Labels converted with label encoding
- Data shuffled to remove ordering bias
- 80:20 split for training and testing

URL FEATURES EXTRACTED

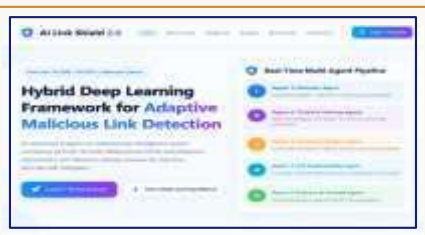
- URL length
- Number of dots
- Number of digits
- HTTPS usage
- Special characters
- Slash count
- URL entropy
- Domain length
- Number of hyphens
- Subdomains
- IP address present
- Suspicious keywords
- Query parameters
- Domain age

EVALUATION RESULTS



Bars scaled 90-100%. Intel Core i5, 8 GB RAM, Windows.

IMPLEMENTATION: WORKING PROTOTYPE



Home: overview of the AI Link Shield 2.0 system



Threat scanner: prediction, confidence and risk



Dashboard: scan statistics and model metrics

TECHNOLOGY STACK

Python 3 · Flask · Random Forest · Neural Network (TensorFlow) · scikit-learn · HTML · CSS · JavaScript · VS Code

CONCLUSION

Integrating Random Forest, Deep Learning and Agentic AI in one framework delivers accurate detection of Benign, Phishing, Malware and Defacement URLs (**98.7% accuracy**) while cutting false predictions. Automated validation, analysis and reporting give users clear, actionable guidance before they open a website.

LIMITATIONS & FUTURE WORK

Limits: accuracy depends on dataset quality and diversity; unseen threats need periodic retraining; deep models need more compute at scale.

Future work: Transformer and Graph Neural Network models, live threat-intelligence feeds, cloud security services, browser extension and mobile apps, and explainable AI (XAI).